

Virtualisierung mit VirtualBox

Allgemeines

VirtualBox ist grundlegend eine Virtualisierungssoftware, welche plattformunabhängig ist. Die Software selbst wird folglich auf einem **Host** installiert und erlaubt es, dass auf diesem Host mehrere unterschiedliche Gastbetriebssysteme in mehreren virtuellen Maschinen betrieben werden. So ist es zum Beispiel möglich, dass auf einem Mac ein Windows und auch ein auf Linux basierendes Betriebssystem installiert wird. Die verschiedenen Gastbetriebssysteme sind dabei losgelöst vom Host. Einzige praktische Limitierung ist die tatsächliche, physische Hardware des Hosts. (vgl. [VBO1])

Warum ist Virtualisierung mächtig?

Es gibt mehrere Gründe, die für eine Virtualisierung sprechen:

Gleichzeitiges Ausführung unterschiedlichster Betriebssysteme

VirtualBox, oder kurz **VBox**, stellt eine Möglichkeit zur Verfügung, welche den parallelen Betrieb mehrere unterschiedlicher Gastbetriebssysteme auf einem Hostbetriebssystem erlaubt. Somit können zum Beispiel Windows Applikationen auf einem Mac betrieben werden, ohne dass der Host einen Dualboot benötigt und neugestartet werden müsste. Aufgrund der Tatsache, dass für jedes Gastbetriebssystem entsprechende Hardwarekriterien spezifiziert werden können, ist man in der Lage auch alte Gastbetriebssysteme zu installieren, selbst wenn die eigene Hosthardware dafür gar nicht mehr unterstützt werden würde. (vgl. [VBO1])

Einfachere Installation benötigter Software

Häufig gibt es Softwarekomponenten, die relativ schwierig zu installieren sind. Softwarehersteller können daher komplette Konfigurationen vorfertigen, die lediglich in eine virtuelle Maschine importiert werden müssen. Somit sind automatisch sämtliche Abhängigkeiten, benötigte Konfigurationen und Komponenten vorhanden. Eine manuelle Schritt für Schritt Einrichtung entfällt dadurch. Man erfährt durch diese Möglichkeit eine erhebliche Zeitersparnis und vermeidet auch potentielle Fehlerquellen, welche innerhalb einer manuellen Installation auftreten können. (vgl. [VBO1])

Einsatz als Testwerkzeug

Sobald eine virtuelle Maschine angelegt wurde, kann diese zu beliebigen Zeitpunkten gestartet, gestoppt, gesichert, kopiert, eingefroren und wieder aufgeweckt werden. Sogar der Transport zwischen unterschiedlichen Hosts ist möglich. Spezielle Snapshots, also gesicherte Zustände einer virtuellen Maschine, erlauben außerdem das Zurücksetzen zu einem gewünschten Stand. Sollte folglich auf einer virtuellen Maschine eine Installation fehlschlagen, Chaos entstehen oder vielleicht aufgrund von Schadsoftware das komplette System zerstört werden, so kann relativ leicht zum gespeicherten Zustand zurückgekehrt werden. (vgl. [VBO1])

Einsparung von Ressourcen

Anstatt mehrere physische Rechner zu betreiben, die jeweils nur eine einzige Aufgabe erledigen, kann man auf einen einzigen Rechner mit mehreren unterschiedlichen virtuellen Maschinen ausweichen. Last kann somit ausgeglichen werden, Hardware- und Stromkosten erheblich gesenkt werden. (vgl. [VBO1])

Hypervisor

Was ist ein Hypervisor?

Ein Hypervisor ist nichts anderes als eine Software, die auf einem Host die Ausführung mehrerer virtuellen Maschinen ermöglicht. Die eigentliche physische Hardware wie CPU, Speicher oder Festplattenkapazität wird vom Hypervisor dabei verwaltet und den unterschiedlichen VMs bereitgestellt. Auszeichnend für den Betrieb von virtuellen Maschinen mit Hilfe eines Hypervisors ist, dass jede virtuelle Maschine ein eigenes, vollumfängliches Betriebssystem mit Anwendungen beinhaltet. Es hat folglich den Anschein, als würde jede VM auf einem eigenen, physischen Rechner laufen. (vgl. [RED1])

Aufgaben des Hypervisors

Verwaltung von Ressourcen

Wie angesprochen, wird die tatsächliche, physische Hardware des Hosts (CPU, RAM, Festplattenkapazität etc.) zwischen den verschiedenen virtuellen Maschinen aufgeteilt. (vgl. [RED1])

Isolation vom Host

Da VMs wie eigenständige Rechner betrachtet werden können, darf ein Problem oder gar ein Absturz einer VM keine Auswirkungen auf den Host oder auf eine andere VM haben. Der Hypervisor isoliert daher jede einzelne VM von sich. (vgl. [RED1])

Hardware Virtualisierung

Der Host verfügt über eine tatsächliche, physische Hardware. Der Hypervisor kümmert sich um die Verteilung der jeweiligen Hardwarekapazitäten. Für die einzelnen VMs wirkt es allerdings so, als ob diese ebenfalls eine eigene physische Hardware besitzen würden. Diese Virtualisierung übernimmt ebenfalls der Hypervisor. (vgl. [RED1])

Optimierung von Performance

Der Hypervisor versucht, dass jede VM möglichst effizient ausgeführt werden kann. Die Leistung einer jeden VM soll grundlegend so nahe wie möglich an der tatsächlichen, physischen Hardware liegen. (vgl. [RED1])

Typen des Hypervisors

Hypervisor Typ 1

Der Hypervisor vom Typ 1 wird auch häufig als Bare Metal Hypervisor bezeichnet und sitzt direkt auf der physischen Hardware des Hosts. Folglich gibt es kein darunterliegendes Betriebssystem. Typisches Beispiel ist Microsoft Hyper-V. (vgl. [RED1]).

Hypervisor Typ 2

Im Gegensatz zum Typ 1, ist ein Hypervisor Typ 2 eine Software bzw. Anwendung, die auf dem entsprechenden Betriebssystem des Hosts installiert wird. Somit nutzt der Typ 2 auch die Ressourcen des Hosts und verteilt die entsprechende Leistung, damit die VMs ausgeführt werden können. Typische Beispiele für Typ 2 sind zum Beispiel VirtualBox oder Parallels. (vgl. [RED1])

VBoxManage

Die VirtualBox besitzt eine grafische Oberfläche, mit der entsprechende virtuelle Maschinen erzeugt werden können. Was allerdings, wenn man dynamisiert bzw. automatisiert VMs erzeugen will? Hier gerät man mit der GUI schnell an seine Grenzen. Unter anderem aus diesem Grund existiert innerhalb der VirtualBox das Kommando

`VBoxManage`. `VBoxManage` ist ein Kommandozeileninterface, welches grundlegend bei einer Windows Installation sich unter nachfolgendem Ordner befindet:

```
C:\Program Files\Oracle\VirtualBox\VBoxManage.exe
```

Mit Hilfe dieses Kommandos kann die ganze Verwaltung von VMs über die Kommandozeile übernommen werden. Um das Kommando über eine Shell überhaupt ausführbar zu machen, muss dieses zunächst in die Umgebungsvariable PATH hinzugefügt werden:

```
$vboxpath="C:\Program Files\Oracle\VirtualBox"

if(-Not ($env:path.split(";") -eq $vboxpath)){ $env:path+="$vboxpath" }
```

IMPORTANT

`$env:path` ist nur für die aktuelle PowerShell Session gültig. Nach einem Schließen des Terminals ist der Pfad folglich innerhalb der Umgebungsvariable nicht mehr vorhanden.

Dauerhaftes Hinzufügen zur Umgebungsvariable

Um den Pfad dauerhaft hinzuzufügen, muss der Pfad in die Umgebungsvariable des Users hinzugefügt werden. Hierfür kann folgender Befehl verwendet werden:

```
$oldpath = [Environment]::GetEnvironmentVariable("Path", "User")
$vboxpath="C:\Program Files\Oracle\VirtualBox"
if(-Not ($oldpath.split(";") -contains $vboxpath)) {
    [Environment]::SetEnvironmentVariable("Path", "$oldpath$vboxpath", "User")
}
```

Andere Möglichkeiten zum Setzen, falls noch nicht geschehen

Weitere Möglichkeiten bzw. Methoden, die nach einem String suchen sind:

- Select-String
- -like

Erste Gehversuche mit VBoxManage

Szenario

Auf einem Netzlaufwerk unter `R:\VBOX-VDIs\Heitzer\student_debian.vdi` befindet sich ein Virtual Disk Image (VDI). Ihr Chef beauftragt Sie, auf Basis dieser VDI zwei neue virtuelle Maschinen einzurichten. Die Konfiguration lautet folgendermaßen:

```
Name: <User>_debian
Basisordner: D:\Daten\<User>\SI\VMs
Memory: 4096
CPUs: 2
```

Da Sie Organisation lieben, legen Sie sich hierfür zunächst einen Ordner für Ihre Festplatten an:

```
mkdir D:\Daten\<User>\SI\Platten
```

Da Sie zwei virtuelle Maschinen benötigen, legen Sie darunter noch zwei Unterordner an:

```
mkdir D:\Daten\<User>\SI\Platten\Original
mkdir D:\Daten\<User>\SI\Platten\Kopie
```

Als nächsten Schritt kopieren Sie die Platte vom Netzlaufwerk in den Ordner `Original` (via Explorer; damit man sieht, wie lange der Kopiervorgang benötigt). Sobald das Kopieren abgeschlossen ist, kopieren Sie über die Kommandozeile Ihr Original in den Ordner `Kopie` :

```
copy D:\Daten\<User>\SI\Platten\Original\student_debian.vdi D:\Daten\<User>\SI\Platten\Kopie\meine_kopie.vdi
```

TERMINAL

Jetzt beginnen Sie über die GUI der VirtualBox die erste Maschine `<User>_debian` zu erzeugen. Sie werden dabei feststellen, dass die Erzeugung problemlos klappt. Im Anschluss starten Sie mit der Erzeugung der zweiten Maschine. Beim Punkt des Hinzufügens der Festplatte bekommen Sie allerdings einen Fehler. Das Plattenabbild für die Kopie konnte nicht geöffnet werden, da die Festplatte anscheinend schon existiert. Innerhalb der Fehlermeldung steht etwas von wegen einer UUID.

Dieser Fehler resultiert darin, dass jede Festplatte eine eindeutige, universelle Identifikation benötigt (= Universally unique identifier / UUID). Da Sie mit dem Kommandozeileninterface `VBoxManage` vertraut sind, wissen Sie natürlich, dass man über die Shell in der Lage ist, eine UUID einer Festplatte abzuändern. Sie führen folglich nachfolgenden Befehl aus:

```
VBoxManage internalcommands sethduuid D:\Daten\<User>\SI\Platten\Kopie\meine_kopie.vdi
```

TERMINAL

Mit Hilfe dieses Befehls wird eine neue UUID für die Platte `meine_kopie.vdi` erzeugt. Im Anschluss kann ohne Probleme eine neue virtuelle Maschine mit der entsprechenden Konfiguration erzeugt werden.

IMPORTANT

Das Kopieren einer vorhandenen VDI, welche schon innerhalb einer VM registriert ist, führt immer dazu, dass dessen UUID schon verwendet wird. Folglich muss für die Kopie eine neue UUID erzeugt werden.

VDI klonen

Um diesem Problem gleich vorweg aus dem Weg zu gehen, gibt es die Möglichkeit, eine Festplatte zu klonen. Hierzu wird ein neues Verzeichnis angelegt:

```
mkdir D:\Daten\<User>\SI\Platten\Klon
```

TERMINAL

Im Anschluss kann die Platte aus dem Ordner `Original` problemlos mit nachfolgendem Kommando geklont werden:

```
VBoxManage clonehd D:\Daten\sheitzer\SI\Platten\Original\student_debian.vdi  
D:\Daten\sheitzer\SI\Platten\Klon\der_klon.vdi --format VDI
```

TERMINAL

Schritt für Schritt zum Skript

Bisher wurden zwei virtuelle Maschinen komplett manuell erstellt. Der Erstellungsprozess über die grafische Oberfläche war dabei allerdings immer der gleiche Ablauf. Man kann somit den ersten Schritt bereits jetzt automatisieren und in ein Skript auslagern:

```
#Pfad zum VBoxManage Command setzen
$vboxpath="c:\Program Files\Oracle\VirtualBox"

if(-Not ($env:path.split(";") -eq $vboxpath)){ $env:path+="$vboxpath"}

#Aktuellen User ermitteln
$curUser = $env:USERNAME

#Namen fuer VM erzeugen
$name = "${curUser}_debian"
$path = "D:\Daten\${curUser}\SI\Platten"
$originalPath = "${path}\Original\student_debian.vdi"
$clonePath = "${path}\Klon\klon_aus_skript.vdi"

#Klonen der Platte
VBoxManage clonehd $originalPath $clonePath --format VDI
```

Erzeugen einer neuen virtuellen Maschine

Im Anschluss muss mit Hilfe des `VBoxManage` Kommandos eine neue virtuelle Maschine erzeugt werden. Hierfür existiert das Kommando `createvm`. Das Kommando hat einen Pflichtparameter `--name`. Mit Hilfe dieses Parameters wird der Name der virtuellen Maschine spezifiziert. Der Parameter `--basefolder` gibt den Basisordner für die virtuelle Maschine an. Neben diesen beiden Parameter ist auch noch der Parameter `--register` von großer Bedeutung. Ohne diesen Parameter wird zwar die virtuelle Maschine erzeugt, allerdings noch nicht innerhalb der VirtualBox registriert, was bedeutet, dass die VM nicht in der Liste der vorhandenen virtuellen Maschinen erscheint:

```
...
$vmPath = "D:\Daten\${curUser}\SI\VMs\${name}"
VBoxManage createvm --name $name --basefolder=$vmPath --register
```

Arbeitsspeicher und CPU Kerne setzen

Um den Arbeitsspeicher auf die gewünschten 4096 MB und die Anzahl der CPU Kerne auf 2 Stück setzen zu können, muss das Kommando `modifyvm` genutzt werden. Mit `modifyvm` können grundlegend die Eigenschaften einer virtuellen Maschine beeinflusst werden. Pflichtparameter für den Befehl ist entweder die UUID oder der Name der VM. Im Anschluss können zum Beispiel Parameter wie `--memory` für das Setzen des Arbeitsspeichers oder `--cpus` für die Anzahl von CPU Kernen verwendet werden:

```
...
VBoxManage modifyvm $name --memory 4096
VBoxManage modifyvm $name --cpus 2
```

Einhängen der geklonten Festplatte

Mit Hilfe von `clonehd` kann eine originale Festplatte mit einer neuen UUID geklont werden. Um nun diese Festplatte in eine neue virtuelle Maschine einhängen zu können, werden die Befehle `storagectl` und `storageattach` benötigt.

`storagectl` dient dazu, dass Speichercontroller konfiguriert und verwaltet werden können. Speichercontroller sind hierbei nichts anderes als virtuelle Komponenten, die als Schnittstelle zwischen der VM und angeschlossenen Speichergeräten wie z. B. Festplatten, CD/DVD Laufwerke etc. dienen. Ein Speichercontroller benötigt dabei immer einen Namen, der mit dem Parameter `--name` angegeben werden kann. Mit der Option `--add` wird ein neuer Speichercontroller hinzugefügt. Jeder Controller hat hierbei immer eine Art (`ide`, `sata`, `scsi` oder `floppysas`). Der genaue Typ eines Speichercontrollers wird über den Parameter `--controller` spezifiziert. Wichtig ist natürlich auch immer, dass bei der Verwendung des Commands wieder entweder die UUID oder der Name der VM angegeben wird:

```
...
VBoxManage storagectl $name --name "SATA" --add sata --controller IntelAhci
```

Via `storageattach` kann im Anschluss die geklonte Festplatte tatsächlich eingehängt werden. Auch hier muss wieder die UUID bzw. der Name der virtuellen Maschine angegeben werden. Für das Hinzufügen der Festplatte benötigt man einen vorher erstellten Storagecontroller, welcher mit Hilfe des Parameters `--storagectl` definiert wird. Der `--port` Parameter gibt an, an welchem Port des Speichercontrollers das Gerät angeschlossen werden soll. Da ein Speichercontroller erlaubt, dass an einem Port mehrere Geräte angeschlossen werden können, gibt es den `--device` Parameter, wodurch die Nummer des Geräts am Port angegeben werden kann. `--typ` gibt den Typen des Speichermediums an. Es gibt folgende Möglichkeiten:

- `hdd` : Für Festplatten (Hard Disk Drives).
- `dvd` : Für CD/DVD-Laufwerke.
- `floppy` : Für Diskettenlaufwerke.
- `disk` : Für andere Arten von Speichermedien (z. B. ISO-Dateien)

Zu guter Letzt wird über `--medium` das entsprechende Speichermedium spezifiziert:

```
...
VBoxManage storageattach $name --storagectl "SATA" --port 0 --device 0 --type hdd --medium $clonePath
```

Netzwerk hinzufügen

Um eine Netzwerkkarte hinzufügen zu können, muss die virtuelle Maschine erneut angepasst werden. Hierfür wird erneut der Befehl `modifyvm` genutzt. Generell existiert der Parameter `--nic<1-N>`. Dieser Parameter steht für Network Interface und erlaubt die Angabe einer natürlichen Zahl von 1 - N. Im Anschluss an diesen Parameter muss der Typ des Network Interfaces spezifiziert werden. Zur Auswahl steht:

- `none`
- `null`
- `nat`
- `bridged`
- `intnet`
- `hostonly`
- `generic`

Um so beispielsweise eine Netzwerkbrücke hinzuzufügen, ist der Nachfolgende Befehl notwendig:

```
...
VBoxManage modifyvm $name --nic1 bridged
```

Für eine Netzwerkbrücke muss auch spezifiziert werden, welche tatsächliche Netzwerkkarte als Basis für die Brücke verwendet wird. Um überhaupt vorhandene Netzwerkkarten mit `VBoxManage` zu identifizieren, kann der allgemeine `list` Befehl verwendet werden. Mit `list` können zum Beispiel allgemein virtuelle Maschinen, Festplatten, Betriebssysteme und viele weitere Informationen angezeigt werden. Mit Hilfe von `VBoxManage list bridgedifs` werden entsprechende physische Netzwerkkarten angezeigt. Um nun die erste Netzwerkkarte innerhalb einer Variable ablegen zu können, ist nachfolgender Befehl zu verwenden:

TERMINAL

```
...
$fistCard = (VBoxmanage list bridgedifs | select-string "^Name" | select-object -first 1) -replace "Name:\s+",
""
```

Die ermittelte Netzwerkkarte muss im Anschluss zur virtuellen Maschine hinzugefügt werden. Dies erreicht man wieder mit dem `modfiyvm` Befehl und dem Parameter `--bridgeadapter<1-N>`. Auch hier gilt wieder, dass beliebig viele Netzwerkbrücken hinzugefügt werden können:

TERMINAL

```
...
VBoxManage modifyvm $name --bridgeadapter1 $fistCard
```

Möchte man einen zweites Netzwerk Interface hinzufügen vom z. B. Typen internes Netzwerk, so sieht der Befehl wie folgt aus:

TERMINAL

```
...
VBoxManage modifyvm $name --nic2 intnet
```

Virtuelle Maschine starten

Sobald diese Grundeinstellungen erledigt wurden, kann die neue virtuelle Maschine auch gleich via Skript gestartet werden:

TERMINAL

```
...
VBoxManage startvm $name
```

Grafikkarte einstellen

Ein sehr wichtiger Schritt innerhalb der automatisierten Auslieferung von virtuellen Maschinen ist immer die Einstellung von Grafikkarten. Auch hierfür kann erneut der Befehl `modifyvm` verwendet werden. Die beiden relevanten Parameter sind `--graphicscontroller` und `--vram`. Mit `--vram` kann der Grafikkarte entsprechender Grafikkartenspeicher zugewiesen werden. Der Parameter `--graphicscontroller` hat Einfluss auf den benutzten Grafikkartencontroller. VBox unterstützt mehrere mögliche Typen:

- `VBoxVGA` : Der eher veraltete Standardcontroller.
- `VMSVGA` : Standardcontroller für Linux und neuere VBox Versionen.
- `VBoxSVGA` : Standardcontroller für Windows.

Um entsprechende Einstellungen durchführen zu können, benötigt man nachfolgenden Befehl:

TERMINAL

```
...
VBoxManage modifyvm $name --graphicscontroller vmsvga --vram 16
```

Im Beispiel wird der Grafikkartencontroller auf `VMSVGA` und der Speicher auf `16 MB` gesetzt.

Anhang

Neue Festplatte erzeugen

Damit via Skript eine komplett neue Festplatte erzeugt werden kann, wird der Befehl `createhd` genutzt. Für die Festplatte muss ein entsprechender Speicherort, die Größe und auch die Art spezifiziert werden. Das kann über die Parameter `--filename`, `--size` und `--format` erzeugt werden:

```
...  
$newPath = "${path}\Neu\neu_aus_skript.vdi"  
VBoxManage createhd --filename $newPath --size 10240 --format VDI
```

Im Anschluss muss die neue Festplatte zur bestehenden virtuellen Maschine hinzugefügt werden. Beim Hinzufügen muss wieder ein Speichercontroller angegeben werden. Hierfür kann der gleiche Controller wie beim Anhängen der geklonten Festplatte genutzt werden:

```
...  
VBoxManage storageattach $name --storagectl "SATA" --port 1 --device 0 --type hdd --medium $newPath
```

Das fertige Skript

Es soll ein fertiges, ausführbares Skript erzeugt werden, dass alle vorherigen Schritte dynamisch zusammenfasst!


```
#Pfad zum VBoxManage Command setzen
$vboxpath="c:\Program Files\Oracle\VirtualBox"

if(-Not ($env:path.split(";") -eq $vboxpath)){ $env:path+="$vboxpath"}

#Aktuellen User ermitteln
$curUser = $env:USERNAME

#Namen fuer VM erzeugen
$name = "${curUser}_debian"
$path = "D:\Daten\${curUser}\SI\Platten"
$originalPath = "${path}\Original\student_debian.vdi"
$clonePath = "${path}\Klon\klon_aus_skript.vdi"
$vmPath = "D:\Daten\${curUser}\SI\VMs"

#Sicherstellen, dass die VBox mit Platte gelöscht wird (Skript muss zur Erstellung öfter ausgeführt werden)
if (VBoxManage list vms | findstr $name) {
    VBoxManage unregistervm $name --delete
}

#Klonen der Platte
VBoxManage clonehd $originalPath $clonePath --format VDI

#Neue VM erzeugen
VBoxManage createvm --name $name --basefolder=$vmPath --register

#Eigenschaften der VM bearbeiten
VBoxManage modifyvm $name --memory 4096
VBoxManage modifyvm $name --cpus 2

#Festplatte einhängen
VBoxManage storagectl $name --name "SATA" --add sata --controller IntelAhci
VBoxManage storageattach $name --storagectl "SATA" --port 0 --device 0 --type hdd --medium $clonePath

#1. Netzwerkkarte hinzufügen
VBoxManage modifyvm $name --nic1 bridged
$fistCard = (VBoxmanage list bridgedifs | select-string "^Name" | select-object -first 1) -replace "Name:\s+",
""
VBoxManage modifyvm $name --bridgeadapter1 $fistCard

#2. Netzwerkkarte hinzufügen
VBoxManage modifyvm $name --nic2 intnet

#Neue Festplatte erzeugen und hinzufügen
$newPath = "${path}\Neu\neu_aus_skript.vdi"
VBoxManage createhd --filename $newPath --size 10240 --format VDI
VBoxManage storageattach $name --storagectl "SATA" --port 1 --device 0 --type hdd --medium $newPath

#Grafikkarte einstellen
VBoxManage modifyvm $name --graphicscontroller vmsvga --vram 16

#Virtuelle Maschine starten
VBoxManage startvm $name
```